

Database Systems

Michelangelo Prisciano

August 4, 2026

Contents

1	Week 1 - Intro To Databases	1
1.1	Database Definition	1
1.2	Relational Databases Rules	1
2	Week 2 - Database Design	2
2.1	Intro	2
2.2	Multiplicity	2
2.3	Partial Participation and Weak Entities	2
2.4	Keys	2
2.5	Joining Tables	3
2.6	Anomalies	3
2.7	Normal forms	3
2.7.1	1NF	3
2.7.2	2NF	3
2.7.3	3NF	4
3	Week 3 - SQL	5
3.1	Intro to SQL	5
3.2	Fundamental Clauses	5
3.3	WHERE Clauses	5
3.3.1	Subqueries	5
3.3.2	BETWEEN	5
3.3.3	LIKE	5
3.4	SELECT Aggregate Functions	6
3.5	Different types of JOIN	6
4	Week 4 - Views, Transactions, Security	7
4.1	Views	7
4.2	Transactions	7
4.3	Integrity	7
4.4	ACID Transaction Properties	8
4.5	Common Inconsistencies	8
4.6	Security	8
5	Week 5 - Distributed Databases	9
5.1	Distributed Systems Definition	9
5.2	Allocation Strategies	9
5.3	Allocation Strategies Comparison	10
5.4	Advantages and Disadvantages	10
5.5	Homogeneous and Heterogeneous DDBMS	10
5.6	Transactions	11
5.7	Transparency	11

5.7.1	Distribution Transparency	11
5.7.2	Location Transparency	11
5.7.3	Fragmentation Transparency	11
5.7.4	Naming Transparency	11
5.7.5	Replication Transparency	11
5.7.6	Transaction Transparency	11
5.7.7	Concurrency Transparency	12
5.7.8	Failure Transparency	12
6	Week 6 - NoSQL Databases	13
6.1	What even is NoSQL?	13
6.2	CAP Theorem	14
6.3	ACID vs BASE	15
6.4	NoSQL Flavours	15
6.4.1	Key-Value Databases	15
6.4.2	Document Databases	16
6.4.3	Node-Edge Graphs	16
6.4.4	Column Families	16
6.5	Summary	16
7	Week 7 - More NoSQL and MongoDB	17
7.1	Replication	17
7.2	Sharding	17
7.3	Aggregation	17
8	Week 8 - NoSQL Data Modelling	18
8.1	Schema Design in NoSQL	18
8.2	Query-Driven Design	18
8.3	Embedding vs. Referencing	18
8.4	The Five Rules	19
8.5	One-to-Many Relationships at Scale	19
8.5.1	Inverted Reference	19
8.5.2	Bucket Pattern	19
8.6	Many-to-Many Relationships	20
8.7	Worked Example: Social Media Application	20
8.8	Design Patterns	20
8.8.1	The Extended Reference Pattern	21
8.8.2	The Subset Pattern	21
8.8.3	The Polymorphic Pattern	22
8.8.4	The Computed Pattern	22
8.9	Schema Design Considerations	23
9	Week 9 - Graph Databases	23
9.1	Review Quiz	24
10	Week 10 - Data Warehousing and Data Mining	25
10.1	From Data to Wisdom	25
10.2	Data Warehousing	25
10.2.1	Key Characteristics	25
10.2.2	ETL Process	25
10.2.3	Data Marts	25
10.2.4	Benefits and Challenges	25
10.3	OLAP and Analytical Operations	26
10.4	Data Mining	26
10.4.1	Key Idea	26

10.4.2 Main Operations	26
10.5 Database Evolution	26
10.6 Summary	26
11 Other Notes	27

1 Week 1 - Intro To Databases

1.1 Database Definition

A database can be defined as any collection of data, or information, that is specially organised for rapid search and retrieval by a computer.

1.2 Relational Databases Rules

The following are two important rules in relational databases design:

- **The Information Rule:** information is represented as data stored in table cells
- **The Guaranteed Access Rule:** each data value must be accessible by a combination of table name + row identifier + column name

2 Week 2 - Database Design

2.1 Intro

In the past information was stored in **hierarchical databases**, which offers a lot of different ways of modelling data inside them. Nowadays the industry switched over to **relational databases**, which at their core ensure that there's only one right way of modelling data. A good design often implements the "*Codd's 12 Rules of Relational Databases*".

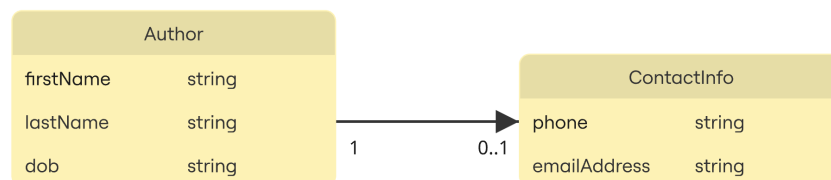
The entity-relationship (ER) diagram helps us modelling a relational database, and to build one we need the following elements:

- **Entities:** these are the types of object we want to store data about, and we can spot these by looking for nouns
- **Attributes:** these are the properties that these objects have
- **Relationships:** we can spot these by looking for the verbs (one-to-one, one-to-many, many-to-many)

2.2 Multiplicity

By setting the **multiplicity** of a relationship we establish the "how many" part of the relationships, for example each recipe book is written by exactly one author but one author can have multiple recipe books (1 for the author, 1..* for the recipe book).

2.3 Partial Participation and Weak Entities



We can see in the example below that the ContactInfo entity might or might not exist in our database, but if it does exist then it's definitely linked to an Author entity. We say that the ContactInfo is a **weak entity**, meaning that it has no independence on its own and relies on another entity for its own existence. We also say that Author has **partial participation** because not every Author *participates* in the relationship with ContactInfo, while the latter has **total participation** because every ContactInfo record must belong to an author.

2.4 Keys

Where are relationships between entities stored then? These are not specifically stored in the database, but they are inferred from **keys**. A **candidate** key is a set of minimal attributes which uniquely identifies each record, a **composite** key consists of two or more attributes, while a **primary** key is a candidate key which is selected to uniquely identify each record. These are the qualities of a good **primary key**:

- Made up of one attribute
- If possible, it should be a numerical attribute
- It should have a meaningful name

There are also situations where a table doesn't have a sensible primary key: in this case we should create our own. We also need to mention that each cell should have only one piece of information in it, aka "lists are bad".

When two tables are related, one table has the primary key of another one. This is called a **foreign key**, and it allows tables to be linked without physically storing the relationship.

2.5 Joining Tables

In a many-to-many relationship (one recipe has many ingredients and one ingredient appears in many recipes, for example), we can't store this relationship in either the Recipe or Ingredient table alone. We need a third table to store relationship-specific data (like QuantityInGrams, as the quantity is specific to the recipe-ingredient pairing, not to the recipe or ingredient independently).

2.6 Anomalies

There's several **anomalies** we can encounter if our ER design is not a solid one. For example:

- Insertion anomaly, when inserting a record depends on information that we don't have
- Update anomaly, when data is duplicated and an update changes only one version
- Deletion anomaly, when deleting some data unintentionally deletes other linked information too

The solution to all of these anomalies is called **normal forms**.

2.7 Normal forms

2.7.1 1NF

First normal form requires that there must be a **primary key**, that each cell contains **only a single value** (no multi-valued attributes or lists), and that there are no duplicate rows or columns. It is the first and essential step in reducing redundancy, improving data integrity, and reducing anomalies in relational database design.

2.7.2 2NF

A relation is in **Second Normal Form** if it is in 1NF and every non-key attribute is **fully functionally dependent** on the primary key. This means that for every primary key value, there is a single unique value for every other attribute in that record.

The key concept is *full functional dependency*: every non-key attribute must depend on the *entire* primary key, not just part of it. A *partial dependency* occurs when a non-key attribute depends only on part of a composite primary key rather than the complete key. For example, in a table with composite primary key (RecipeName, IngName), if the Origin attribute depends only on RecipeName and ignores IngName, this is a partial dependency that violates 2NF. Note that if a primary key consists of only one attribute, partial dependencies cannot occur.

2.7.3 3NF

A relation is in **Third Normal Form** if it is in 1NF and 2NF, and there are no **transitive dependencies**. A transitive dependency occurs when a non-key attribute depends on another non-key attribute rather than directly on the primary key. In other words, an attribute depends on another attribute that is not part of the primary key, creating an indirect dependency chain.

Given:

$$\text{EmployeeID} \rightarrow \text{DepartmentID}$$
$$\text{DepartmentID} \rightarrow \text{DepartmentName, ManagerID}$$

We get a **transitive dependency**:

$$\text{EmployeeID} \rightarrow \text{DepartmentID} \rightarrow \text{DepartmentName, ManagerID}$$

To achieve 3NF, such dependencies must be eliminated by separating the data into distinct tables, ensuring each non-key attribute depends only on the primary key.

3 Week 3 - SQL

3.1 Intro to SQL

SQL (Structured Query Language) is the language we use to manage structured data in a relational database management system. SQL allows us to execute CRUD operations, aka read, create, update and delete. Different databases implement SQL in different way, which led to the creation of MySQL, SQLite, PostgreSQL.

3.2 Fundamental Clauses

There's a set of fundamental clauses used to write SQL, these are:

1. **SELECT** → which columns appear in the output?
2. **FROM** → which table(s) are we looking up?
3. **WHERE** → what are the filters?
4. **GROUP BY** → how do we chunk our data together?
5. **HAVING** → what are the filters for the groups?
6. **ORDER BY** → how do we order the output?
7. **LIMIT BY** → how many records do we want?

3.3 WHERE Clauses

There's different types of operators we can use after **WHERE**.

3.3.1 Subqueries

This is a subquery example.

```
SELECT Name, Milliseconds
FROM Track
WHERE Milliseconds >
    (SELECT AVG(Milliseconds) FROM Track);
```

EXISTS and NOT EXISTS are for use only with subqueries and it produces a true/false result.

```
Select Title FROM Album a
WHERE EXISTS(
    SELECT 1 FROM Track t
    WHERE t.AlbumId = a.AlbumId
    AND t.Milliseconds > 60000
);
```

3.3.2 BETWEEN

```
SELECT Name, (Milliseconds / 1000) AS Minutes
FROM Track
WHERE (Milliseconds / 60000) BETWEEN 9 AND 10;
```

3.3.3 LIKE

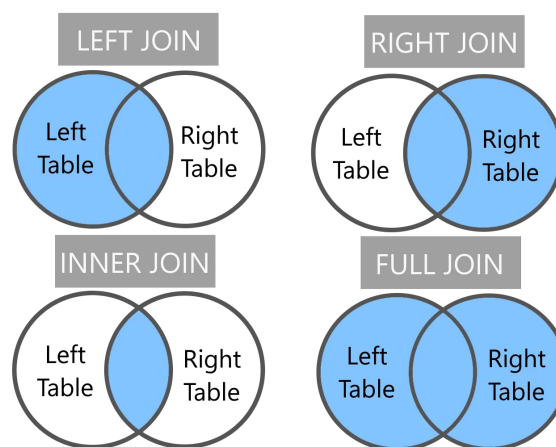
```
SELECT Name
FROM Track
WHERE Name LIKE "%Love%"
```


3.4 SELECT Aggregate Functions

We also have five main types of SELECT aggregate functions:

- **COUNT** → returns the number of values in the specified column
- **SUM** → returns sum of values in specified column
- **AVG** → returns average of values in specified column
- **MIN** → returns smallest value in specified column
- **MAX** → returns largest value in specified column

3.5 Different types of JOIN



4 Week 4 - Views, Transactions, Security

4.1 Views

In SQL a **view** is a virtual table obtained from a query that lives within the query. It doesn't exist in the database, however if we wanted to update a tables in the database we could also use a view.

```
CREATE VIEW CityPopulation AS (<query>);
```

A view can be thought of as something like a layer that hides the complexity of the original query, something like an API. Views are also important for security as they can be used to restrict the amount of information available to see. In particular, a **vertical view** is one that selects only some columns (example below).

```
CREATE VIEW DoctorView AS SELECT
    (PatientNo,
     Surname,
     Telephone) FROM Patients;
```

An **horizontal view** is one that selects only some rows (example below).

```
CREATE VIEW DoctorView AS SELECT
    (PatientNo,
     Surname,
     Telephone,
     WhateverElse)
FROM Patients
WHERE PatientNo = '1234';
```

4.2 Transactions

A **transaction** is a sequence of SQL statements that always translate the database from one consistent state to another one. If a transaction is successful, the changes will commit to the database otherwise if any sort of error is encountered the transaction will rollback.

4.3 Integrity

An important concept in database design is **integrity**, which is about the accuracy and reliability of the data. We can define different types of integrity:

- **Domain Integrity:** this means that all attributes of a record correspond to their expected data type. We cannot put words into an INT data type, for example
- **Entity Integrity:** this means that each row in a table should have a unique and not-NULL primary key value. No attribute of the primary key can be NULL either
- **Referential Integrity:** in general FK values must match a PK in their 'home table'. We should think of this type of integrity when the PK value changes or when the PK value is removed. There's different ways of dealing with this, either with "no action" or with "cascading" the change e.g. remove all albums belonging to the artist if the artist is deleted
- **User Defined Integrity:** this means that we can write our custom SQL rules for other constraints that we have

4.4 ACID Transaction Properties

ACID is an acronym that refers to the set of four key properties that define a transaction:

- A: Atomicity. either all operations in a transaction are successful, or none of them are
- C: Consistency. No integrity constraints are violated
- I: Isolation. Transactions should execute as if the transactions were running one after the other
- D: Durability. Once a transaction has committed, it is not lost due to any subsequent failure

A transaction is a **LUW** (logical/least unit of work), which means that either everything is completed, or rolled back. The syntax for a transaction includes statements such as **START TRANSACTION** (entry point of a LUW), **COMMIT** (end point that makes the changes in LUW permanent), **ROLLBACK** (undoes all uncommitted changes in LUW back to the **START**).

4.5 Common Inconsistencies

Some of the common issues that can be encountered when dealing with databases are **concurrency control**, **lost update problem**, **uncommitted dependency problem**, the **inconsistent analysis problem**, the **deadlock problem**.

4.6 Security

Some of the worst causes of security breach in a database are poor database configuration and SQL injection. The latter can be mitigated by using **prepared statements** and **input sanitisation**.

5 Week 5 - Distributed Databases

5.1 Distributed Systems Definition

A distributed database is formally defined as a logically inter-related collection of shared data, which is **physically distributed** over a computer network. In a nutshell, data is all related like we've been talking so far, but we put it in different places for various reasons (for example, cost, availability, latency, no single point of failure).

Another key important point of distributed databases is that from the point of view of the user there should be no difference between using databases and distributed databases.

So, instead of keeping your entire database in one location, we:

1. **Fragment (split) the data** → Break the database into smaller pieces, like taking individual chocolates out of the box. Each fragment contains a subset of your data (maybe rows, columns, or logical partitions)
2. **Replicate (copy) fragments** → Some fragments might be duplicated across multiple locations. This provides redundancy and improves read performance
3. **Allocate to sites** → Distribute these fragments/copies to different physical locations world-wide. Each site might store different fragments
4. **Connect via network** → All sites communicate through a network. Allows coordinated queries across fragments and enables synchronization between replicas

Fragmentation is called **horizontal** when we split tables by record, **vertical** when we split up tables by attributes and **hybrid** when it's a bit of both.

5.2 Allocation Strategies

The following are the most important **allocation strategies**:

- **Centralised**: single database and DBMS stored at one site
- **Partitioned**: Database is partitioned into disjoint fragments and each fragment is assigned to one site.
- **Complete Replication**: maintain a complete copy of the database at each site.
- **Selective Replication**: combination of partitioning, replication and centralisation.

5.3 Allocation Strategies Comparison

Below we can see a comparison between the various strategies:

Strategy	Locality of Reference	Reliability & Availability	Performance	Storage Costs	Communication Costs
Centralised	Lowest	Lowest	Unsatisfactory	Lowest	Highest
Partitioned	High	Low for item; high for system	Satisfactory	Low	Low
Complete Replication	Highest	Highest	Best for read	Highest	High for update; low for read
Selective Replication	High	OK for item; high for system	Satisfactory	Average	Low

Table 1: Comparison of distributed database strategies

"Locality of Reference" means how close the data is to the users/applications that need to access it. Higher locality = data is physically closer to where it's being used.

5.4 Advantages and Disadvantages

General advantages and disadvantages of distributed systems.

Advantages	Disadvantages
Reflects organisational structure	Complexity (of design and of the final system)
Shareability of data with local autonomy	Increased cost
Availability (multiple sources)	Security control is more difficult
Reliability (built in redundancy)	Integrity control is more difficult
Performance (multiple processors)	Data communications are subject to external factors
Modular growth is easier	

Table 2: Advantages and disadvantages of distributed databases

5.5 Homogeneous and Heterogeneous DDBMS

Homogeneous DDBMS	Heterogeneous DDBMS
All sites use the <i>same</i> DBMS software	<i>Not all</i> sites use the same DBMS software
Easiest to design and manage	Typically because of legacy systems
Approach provides incremental growth and allows increased performance	(Generally, we would not design a new system this way)

Table 3: Homogeneous vs. Heterogeneous DDBMS

5.6 Transactions

A **transaction** is formally defined as an action, or series of actions, carried out by a single user or application program, that reads or updates the contents of the database.

In a distributed database system, **distributed transaction** management refers to a system where a single transaction accesses data stored across more than one location. To handle this, the transaction is divided into several sub-transactions, with each one corresponding to a specific site and typically managed by an agent. Despite being split, the transaction must remain indivisible as a whole, which is essential for maintaining atomicity—meaning it either fully completes or does not happen at all. Additionally, a distributed database management system (DDBMS) must ensure that each individual sub-transaction is also indivisible to preserve the integrity of the overall process.

5.7 Transparency

Transparency of function is the key to the operation of distributed databases. After creating the complex distributed structure, we want to hide it all (and its inherent problems) from the user, as for them the system still has the look and feel of a local database (all the complexity is transparent to the user). We'll now see different types of transparencies.

5.7.1 Distribution Transparency

It allows the user to perceive the database as a single, logical entity.

5.7.2 Location Transparency

Users don't need to know where the data is stored, they're unaware.

5.7.3 Fragmentation Transparency

Users don't need to know that data has been split.

5.7.4 Naming Transparency

Each item in the database has a unique logical name as they don't need to know physical site or network address. In other words, the user sees only a logical name for a resource — not its physical location (server, disk, network address) as data is distributed.

5.7.5 Replication Transparency

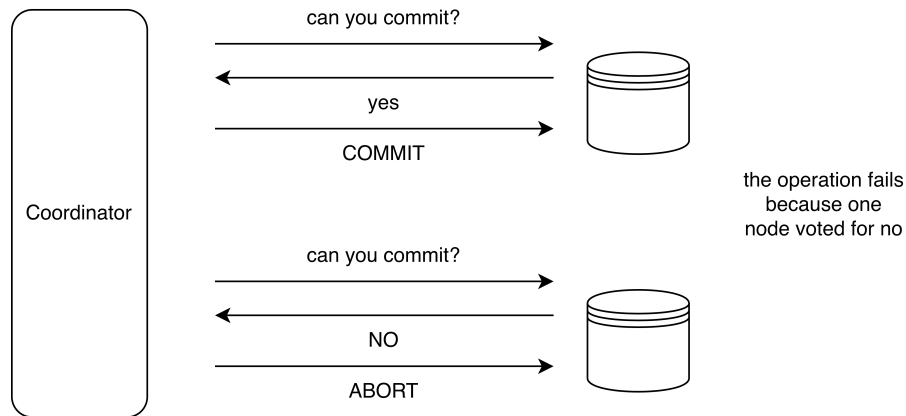
Users don't need to know that data is replicated.

5.7.6 Transaction Transparency

It ensures that all distributed transactions maintain the distributed database's integrity and consistency. Each distributed transaction accesses data stored at more than one location, therefore Each transaction is divided into several sub-transactions, one for each site to be accessed, and the DDBMS must ensure the indivisibility of both the global transaction and each of the sub-transactions.

5.7.7 Concurrency Transparency

All transactions must execute independently and be logically consistent with results obtained as if transactions executed one at a time. The two phase commit is an approach used to ensure consistency.



5.7.8 Failure Transparency

DDBMS must ensure atomicity and durability of all global transactions. Hence ensure that sub-transactions of global transactions either all commit or all rollback. Thus, DDBMS must synchronise global transactions to ensure that all sub-transactions have completed successfully before recording a final COMMIT for the global transaction.

6 Week 6 - NoSQL Databases

6.1 What even is NoSQL?

SQL is cool, but is that the best way of dealing with big data in all scenarios? The answer is no, as we'll see shortly. To remind ourselves, these are the five Vs which are the defining characteristics of Big Data.

1. **Volume:** handling large quantities of data
2. **Velocity:** dealing with the speed that new data comes in
3. **Variety:** integrating the information from various sources
4. **Value:** extract information (wisdom) from the integrated sources
5. **Veracity:** figuring out what information is real and what is not

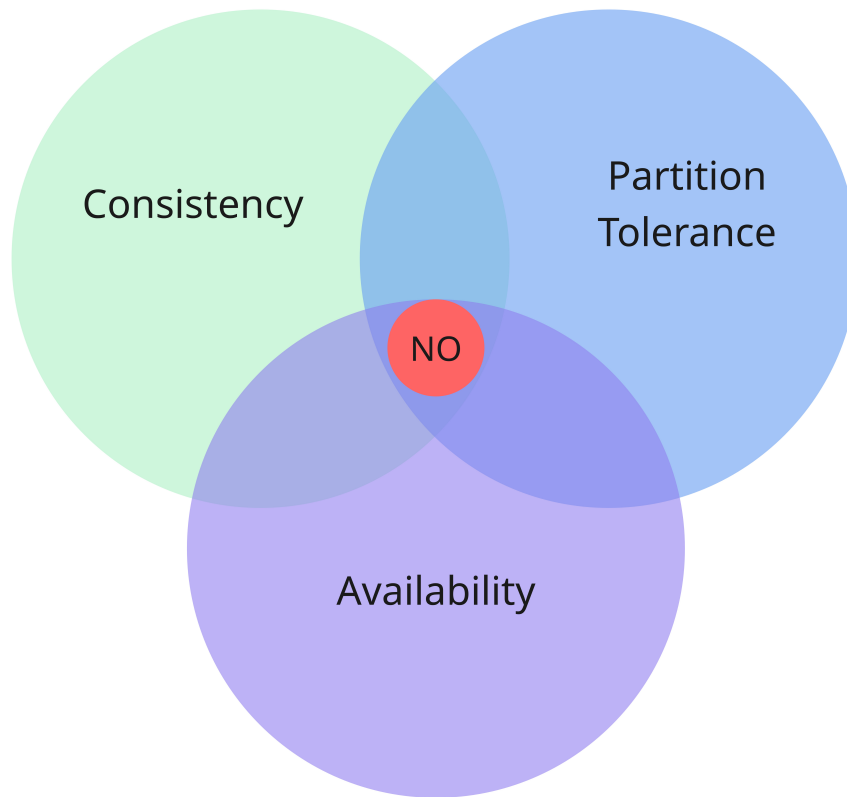
Following we can find a comparison between vertical and horizontal scaling.

Vertical Scaling (Scaling UP)	Horizontal Scaling (Scaling OUT)
CPU and RAM resource upgrades to a single server	Add more machines to distribute the load
Less complexity	Infinitely scalable
There's a limit	Gets a bit complicated

Table 4: Homogeneous vs. Heterogeneous DDBMS

NoSQL databases are focused on **distributed** and **horizontal scalability**, and the aim is to spread out data and make replication easier. It's important to think carefully about our goal: for example, it would not be ideal designing a banking system on NoSQL because of the inner weaker integrity constraints.

6.2 CAP Theorem



The CAP Theorem states that any distributed data store can provide at most two of the following three guarantees:

- **Consistency:** All clients see the same data at the same time, no matter what node they connect to.
- **Partition Tolerance:** The system keeps working even if network issues occur.
- **Availability:** Every client will get a response even if 1+ nodes are down.

If you demand **Consistency** (every read gets the most recent write) and **Partition Tolerance** (the system keeps working despite network breaks), you must sacrifice **Availability** (some nodes may refuse to respond).

By choosing **Availability** (every node always responds) and **Partition Tolerance** we have to sacrifice **Consistency**. Different nodes might return different data during a network partition, but the system stays up. This is an acceptable trade-off for many real-world systems (for example, DNS).

Finally, by choosing **Consistency** and **Availability** we then sacrifice **Partition Tolerance** (which we're not interested about since we're dealing with distributed systems).

6.3 ACID vs BASE

In the previous section we saw that CAP says that during a network partition, we must choose C (Consistency) or A (Availability). If we choose:

- **C(onsistency)** → We are building an ACID-like system (CP type). It stops responding during partitions to avoid errors.
- **A(vailability)** → We are building a BASE-like system (AP type). It keeps responding during partitions, accepting that data might be temporarily wrong (inconsistent), but promises to fix it "eventually"

BASE stands for:

- **Basically Available:** systems remains generally available. Even if certain nodes or components fail, there's some level of service
- **Soft State:** there's allowance for temporary inconsistencies
- **Eventually consistent:** does achieve consistency over time, rather than immediately. After a time, all replicas or nodes in the system will eventually converge to a consistent state.

6.4 NoSQL Flavours

NoSQL is a schema-flexible design with no predefined data model, therefore we have lots of flexibility. Data isn't stored in tables, instead it might be stored as one of the formats in the subsequent sections.

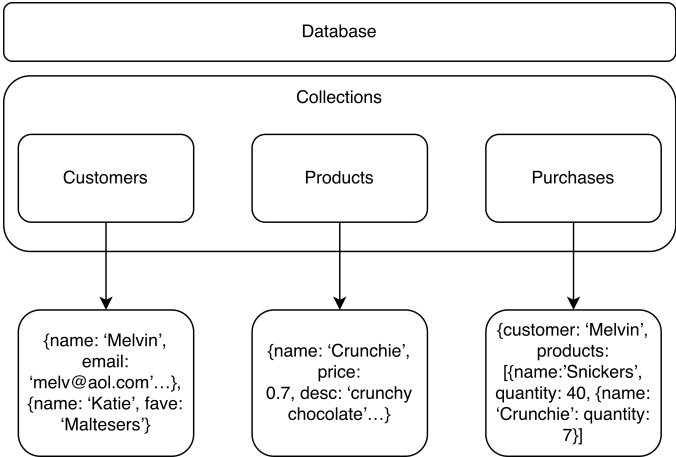
6.4.1 Key-Value Databases

In this case we have keys, which are mapped to values through a hash function. This type of database is good for sessions authentication, caching, real-time updates. Redis is an in-memory database, for example.

Advantages	Disadvantages
Simple, we just need a key and a value	We can't search by value, we can only get data by the key
Flexible, values can be anything	This is good for quick look-ups and writes only
Fast, stored in memory for real time access	There's no easy way to join data together

6.4.2 Document Databases

These are stored in a semi-structured format, and we have entire documents stored as records. These are a set of key-value pairs, we can have nested data types and a flexible schema (fields don't have to match across records), and more crucially we can search by value.



6.4.3 Node-Edge Graphs

This is a database with an explicit graph structure, with **entities** and **relationships**. It's a good choice when designing a system that requires path queries and/or semantic links (train/airline booking, social media connections, product recommendation, etc).

6.4.4 Column Families

This type of database solves the problem of having to store NULL values when not all rows contain the same data. In simple terms, a column family is a flexible table where each row can have a different set of columns. Unlike traditional relational databases where all rows in a table have the same structure, column families allow each row to store only relevant data, therefore reducing redundancy.

Advantages	Disadvantages
Flexible schema, as new columns can be added	No multi-row transactions
Each row can have different columns	
Related columns stored together for quick retrieval	

6.5 Summary

To sum up, SQL is about "how do I stored this data" while NoSQL is about "how do I access this data".

7 Week 7 - More NoSQL and MongoDB

The **golden rule** in NoSQL is that data that is accessed together, should be stored together.

7.1 Replication

Replication means making copies of data to maximise availability. To achieve this, MongoDB uses replica sets, which are secondary nodes where data is replicated from the primary node. The nodes send each other a "heartbeat" message to make sure they're all doing fine. If the primary node goes down, an **election** is held for a new primary node. We can see that **data redundancy** is a good thing, because through it we get high availability.

In a standard scenario, the primary node is very busy as it deals with all reads and writes. We can also lighten the load by giving clients a read preference, which can allow them to read from secondary nodes.

7.2 Sharding

Sharding means splitting data across multiple machines for added scalability, performance and storage capacity. It's done by dividing the data into smaller partitions (shards), and each "shard" is on a separate server/node.

The **shard key** is one (or more fields) that decide on what shard is the document stored. Choosing the right field for this is crucial in order to prevent the following:

- Hot partition: when one server has an unusually high load
- Low cardinality: where the shard key values are too limited (example, true and false) leaving other shards with nothing on them
- Biased key frequency: when cardinality is fine, but distribution is not, as one node has substantially more documents on it compared to the others.
- Monotonically increasing keys (for example, date): this works fine for a while, but it's bad in the long run.

7.3 Aggregation

Aggregation is the process of multi-stage querying for more complex analysis.

8 Week 8 - NoSQL Data Modelling

8.1 Schema Design in NoSQL

A common misconception is that NoSQL databases are *schema-absent*. In reality, they are **schema-flexible**: the schema is not enforced at the database level, but a data model still exists and must be deliberately designed.

8.2 Query-Driven Design

NoSQL schema design follows the opposite philosophy to relational databases. In SQL, the schema is designed first and queries are written to fit it. In NoSQL, **queries are identified first**, and the schema is designed around them — this is known as **query-driven design**.

Before modelling, it is worth asking:

- What does my application do?
- What data will I store, and what does it look like?
- What data is most valuable?
- How will users access the data, and what access patterns will emerge?
- What are the relationships amongst the data?

The **golden rule** of NoSQL data modelling follows directly from this:

Data that is accessed together should be stored together.

8.3 Embedding vs. Referencing

When modelling one-to-many relationships in a document database, there are two primary strategies.

Embedding stores related data directly inside the parent document:

```
db.authors.updateOne({first_name: "Keith"}, {$set: {books: [
  {title: "Cognac Cookery",    category: "French"},
  {title: "Hot & Spicy Floyd", category: "Spicy"},
  {title: "Floyd on Italy",    category: "Italian"}
]}})
```

Referencing stores only the identifiers of related documents, which live in a separate collection:

```
db.authors.updateOne({first_name: "Keith"}, {$set: {books: [
  "0146003527", "0146001338", "0718138104"
]}})
db.books.insertOne({"_id": "0146003527", title: "Cognac Cookery"})
```

A join (via \$lookup) is then required to retrieve the full data:

```
db.authors.aggregate([
  {$match: {first_name: "Keith"}},
  {$lookup: {from: "books", localField: "books",
    foreignField: "_id", as: "book_info"}},
  {$project: {book_info: 1}}
])
```

The trade-offs between the two approaches are summarised below:

As a general principle, embeddings favour **read performance**, while referencing favours **write performance**.

Property	Embedding	Referencing
Single query to retrieve all data	✓	×
No data duplication	×	✓
Smaller documents	×	✓
Single operation to update/delete	✓	×
Requires joining multiple documents	×	✓
Risk of large/bloated documents	✓	×

Table 5: Trade-offs between embedding and referencing

8.4 The Five Rules

1. **Favour embedding unless there is a strong reason not to.** NoSQL databases are not optimised for joins. Embedding avoids the overhead of `$lookup` operations and allows all relevant data to be fetched in a single query.
2. **The need to access an object on its own is a strong reason for not embedding.** If a subdocument must be queried independently (e.g. books on a retail platform where price and stock change frequently), it warrants its own collection.
3. **Avoid lookups if they can be avoided** (but consider them when they result in a better schema design). Lookups are expensive; however, they may be the right trade-off if they prevent excessive data duplication or document bloat.
4. **Arrays should not grow without bounds.** MongoDB documents have a 16 MB size limit. Embedding an ever-growing array (e.g. sensor readings) will eventually cause problems. For high-cardinality one-to-many relationships, the reference should be placed on the “many” side, or the **Bucket Pattern** should be used (see below).
5. **How you model your data depends entirely on your application’s access patterns.** There is no universal schema; the correct design is always application-specific.

8.5 One-to-Many Relationships at Scale

When one document can have millions of related documents (e.g. a sensor and its readings), even referencing breaks down if all IDs are stored in an array on the parent:

```
// Problematic: unbounded array on parent
{"sensor_id": "abc123", "readings": [123439, 123440, ...]}
```

Two solutions exist:

8.5.1 Inverted Reference

Place the reference on the “many” side instead:

```
// sensor document
{"_id": "abc123", "location": "Dundee"}

// reading document
{"_id": 123439, "sensor_id": "abc123",
 "timestamp": "2026-02-22T10:01:00", "type": "temperature", "value": 12.5}
```

8.5.2 Bucket Pattern

Group readings into time-bounded buckets. Each bucket document covers a fixed interval and stores an array of measurements, along with precomputed aggregates:

```
{
  "sensor_id": "abc123",
  "start_date": ISODate("2026-02-22T10:00:00Z"),
  "end_date": ISODate("2026-02-22T10:59:59Z"),
  "measurements": [
    { "timestamp": "...", "type": "temperature", "value": 12.5 },
    ...
  ],
  "values": 42,
  "sum_temp": 1300.9
}
```

This keeps document size bounded while allowing efficient range queries and aggregation.

8.6 Many-to-Many Relationships

Document databases are **not optimised** for many-to-many relationships. Where they are unavoidable, the standard approach is to hold references on both sides. For example, in a recipe database where recipes share ingredients:

```
// recipe document
{
  "_id": 123, "name": "Chocolate cake",
  "ingredients": [{ing_id: 1, quantity: "225g"}, {ing_id: 2, quantity: 4}]
}

// ingredient document
{
  "_id": 1, "name": "Flour", "costPer100g": 0.10, "recipes": [123, 456]}
}
```

8.7 Worked Example: Social Media Application

To illustrate Rule 5, consider a social media platform where the access patterns are:

Type	Operation	Frequency / day
Read	Read post	100,000,000
Write	Author post	1,000
Write	Make comment	10,000
Write	Give like	100,000

Table 6: Access patterns for a social media application

Since reads vastly outnumber writes, the schema should lean towards embedding to minimise query latency. However, unbounded growth and duplication must still be managed. A practical design might:

- Embed only the author attributes needed inside the post (e.g. `id`, `name`), rather than the full user document.
- Embed a bounded subset of the most recent comments (e.g. 1–5), and reference the remainder in a separate `comments` collection.
- Keep individual like documents in a separate collection to avoid unbounded array growth, but store a `total_likes` counter directly in the post document for fast display.

This hybrid approach reflects the core tension in NoSQL schema design: **embeddings are best for reads, referencing is best for writes**.

8.8 Design Patterns

The social media example above naturally motivates four named patterns that appear frequently in MongoDB schema design.

8.8.1 The Extended Reference Pattern

When a document references another collection, a `$lookup` join is required to retrieve associated fields. If only a small number of those fields are needed and they change infrequently, it is more efficient to **embed those fields directly** inside the referencing document alongside the foreign key — avoiding the join entirely for the common case.

For example, rather than storing only `authorid` in a post and joining to the `users` collection to get the author's name and role, we embed the stable attributes:

```
db.posts.update({postid: 1}, {$set: {
  author: {
    authorid: 1,
    name:     "Daniel Rough",
    role:     "Lecturer at University of Dundee"
  }
}})
```

The full user document still lives in the `users` collection; the embedded fields act as a de-normalised cache of the most-needed attributes. If the user's name or role ever changes, the embedded copy must also be updated, so this pattern is best suited to fields that are **unlikely to change often**.

8.8.2 The Subset Pattern

When a one-to-many relationship could produce a very large embedded array (e.g. all comments on a post), but only a small subset of those documents is displayed by default, embedding *all* of them is wasteful. The Subset Pattern addresses this by:

1. Keeping **all** related documents in their own collection (e.g. `comments`).
2. Embedding only the most relevant subset (e.g. the 3 most recent comments) directly inside the parent document as a duplicate.

The embedded subset is populated and refreshed with an aggregation pipeline using `$lookup`, `$sort`, `$limit`, and `$merge`:

```
db.posts.aggregate([
  { $lookup: {
    from: 'comments',
    let: { id: '$postid' },
    pipeline: [
      { $match: { $expr: { $eq: ['$postid', '$$id'] } } },
      { $sort: { date: -1 } },
      { $limit: 3 }
    ],
    as: 'recentComments'
  }},
  { $merge: {
    into: 'posts',
    whenMatched: 'replace',
    whenNotMatched: 'discard'
  }}
])
```

This keeps the post document small while still allowing the UI to display a preview of comments without an additional query.

8.8.3 The Polymorphic Pattern

Because MongoDB does not enforce a fixed schema, documents within the same collection **do not need identical fields**. The Polymorphic Pattern exploits this: objects that share more similarities than differences can be stored together in a single collection, with each document carrying only the fields relevant to its type.

A practical example is a `likes` collection that stores likes on both posts and comments. Rather than maintaining two separate collections, a single `likes` collection holds documents that differ only in which foreign key they carry:

```
// likes on a post
{postId: 1, userid: 5, type: "like"}
{postId: 1, userid: 6, type: "love"}

// likes on a comment (same collection)
{commentid: 2, userid: 1, type: "like"}
{commentid: 4, userid: 2, type: "love"}
```

The same pattern applies more broadly: a `books` collection at a retailer might store physical books, e-books, and audiobooks together, each with a different set of fields, while sharing common attributes like `title`, `price`, and `author`.

8.8.4 The Computed Pattern

When an aggregated value (e.g. a like count, an average rating, a running total) is displayed alongside a document every time it is read, recomputing it on every read is expensive. The Computed Pattern avoids this by **pre-computing and storing the value** inside the document, refreshing it periodically or on write rather than on every read.

For the social media example, the total number of likes per post can be computed by joining the `likes` collection and grouping by reaction type, then merging the result back into the post:

```
db.posts.aggregate([
  { $lookup: {
    from: "likes",
    localField: "postId",
    foreignField: "postId",
    pipeline: [
      { $group: { _id: "$type", count: { $sum: 1 } } }
    ],
    as: "total_likes"
  }},
  { $merge: {
    into: "posts",
    whenMatched: "replace",
    whenNotMatched: "discard"
  } }
])
```

The result is a `total_likes` array embedded directly in each post document, ready to be displayed without any additional aggregation at read time. This trades **exact real-time accuracy** for **read performance** — acceptable in most social media contexts where a slightly stale like count is not critical.

Other common uses of the Computed Pattern include: follower/connection counts, average product ratings, remaining event seats, running revenue totals, and precomputed min/max/average sensor readings.

8.9 Schema Design Considerations

When designing a NoSQL schema in practice, the following questions help drive the right decisions:

- **Scale:** How many documents of each type will exist? A design that works for hundreds of users may break for millions. The expected cardinality of each relationship directly affects whether embedding or referencing is appropriate.
- **Pre-computation:** Which aggregated values are displayed frequently enough to justify storing them? Counts, averages, and sums are good candidates.
- **Justification:** Every modelling decision — embedding, referencing, separate collection, denormalisation — should be justified by the access patterns of the application.
- **ER diagrams:** Even in a document database context, sketching an ER diagram of entities, attributes, and relationship types (one-to-one, one-to-many, many-to-many) is a useful first step before committing to a schema.

9 Week 9 - Graph Databases

Graph databases are great for anything that looks like a network, ie where the relationships between entities are as important as the entities themselves.

	Relational Database (MySQL)	Aggregation	Graph Database
Optimised for:	Structured data storage	Clumping data together	Storing connections between elements
Multi-hop queries:	Lots of table joins – gets rubbish very quickly	Really good at this sort of 'aggregate' operation	Ideal for this – scales up no problem
Same operation on lots of elements:	(Strong performance)	(Good performance)	Nowhere near as efficient – not optimised

Table 7: Comparison of database types for different operations

Because the graph database only looks at records directly connected to other records, irrelevant records aren't visited, which is what makes the searching so much faster.

In summary, these are the characteristics of graph databases:

- Really useful when we care about relationships between entities
- High flexibility compared to relational databases
- Not great for finding all the entities with a certain property, or for bulk-editing of entities in general

9.1 Review Quiz

10 Week 10 - Data Warehousing and Data Mining

10.1 From Data to Wisdom

Modern systems have shifted from storing small, transaction-focused datasets to collecting vast amounts of heterogeneous data, often referred to as **Big Data**. The goal is to transform:

Data → Information → Knowledge → Wisdom

This progression enables organisations to discover patterns and make predictions about future events.

10.2 Data Warehousing

A **data warehouse** is defined (Inmon, 1993) as:

“A subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management’s decision-making process.”

10.2.1 Key Characteristics

- **Subject-oriented:** Organised around core business entities (e.g., customers, sales), not applications.
- **Integrated:** Combines data from multiple heterogeneous sources into a consistent format.
- **Time-variant:** Stores historical data as snapshots over time, enabling trend analysis.
- **Non-volatile:** Data is stable; once stored, it is not overwritten but appended.

10.2.2 ETL Process

Data warehouses are built using the **ETL process**:

- **Extract:** Collect data from various sources.
- **Transform:** Clean and standardise data.
- **Load:** Store data into the warehouse.

10.2.3 Data Marts

A **data mart** is a subset of a data warehouse tailored to specific business units (e.g., sales or marketing). Architectures include:

- Top-down (warehouse first, then marts)
- Bottom-up (marts first, then integrated)

10.2.4 Benefits and Challenges

Benefits:

- Improved decision-making
- Competitive advantage
- Better use of existing data

Challenges:

- High cost and complexity
- Data integration issues
- Organisational and terminology conflicts

10.3 OLAP and Analytical Operations

Online Analytical Processing (OLAP) enables complex analysis of warehouse data. Key operations include:

- **Roll-up:** Aggregating data (e.g., daily to monthly).
- **Drill-down:** Accessing more detailed data.
- **Slice and Dice:** Viewing data from different perspectives.
- **Pivot:** Rotating data for alternative visualisations.

10.4 Data Mining

Data mining is the process of extracting previously unknown, useful, and actionable knowledge from large datasets using techniques from statistics and machine learning.

10.4.1 Key Idea

Unlike OLAP (which answers known questions), data mining discovers **unknown patterns** and relationships.

10.4.2 Main Operations

- **Predictive Modelling:**
 - Uses supervised learning (training and testing phases)
 - Includes classification and value prediction
- **Database Segmentation:**
 - Uses unsupervised learning
 - Groups data into clusters with similar characteristics
- **Link Analysis:**
 - Finds relationships between data (e.g., association rules)
 - Includes sequential pattern discovery
- **Deviation Detection:**
 - Identifies anomalies or outliers
 - Useful for fraud detection and quality control

10.5 Database Evolution

- **OLTP:** Real-time transaction processing (current state of data)
- **Data Warehouse:** Historical, integrated data storage
- **OLAP:** Analytical querying and reporting
- **Data Mining:** Discovery of hidden patterns and predictive insights

10.6 Summary

Data warehousing enables the storage and organisation of large-scale historical data, while OLAP provides analytical tools to explore it. Data mining extends this further by uncovering hidden patterns and supporting predictive decision-making, forming a critical component of modern business intelligence systems.

11 Other Notes